

# BoxAdmin — Fase 0: Fundamentos

## Especificación técnica para agente de desarrollo (Claude Code)

Septiembre 2026

### Contexto del proyecto

**BoxAdmin** es un sistema de gestión para gimnasios y estudios (Pilates, Yoga, funcional, etc.), multi-tenant desde el diseño, pensado para reemplazar y mejorar las funcionalidades de TurnoFit. Este documento especifica **exclusivamente la Fase 0**: la base técnica sobre la que se construyen todas las fases siguientes.

**Objetivo de esta fase:** dejar corriendo el esqueleto completo del proyecto (backend, base de datos, cache, autenticación con roles, aislamiento por tenant) sin todavía implementar lógica de negocio del gimnasio (eso empieza en la Fase 1).

**Criterio de éxito:** un desarrollador puede levantar el entorno con un solo comando, registrar un tenant de prueba, loguearse, y recibir un JWT válido cuyos requests quedan automáticamente filtrados por `tenant_id`.

---

## 1. Stack e instalación

### 1.1 Requisitos previos del entorno

- Node.js 20 LTS
- Docker + Docker Compose v2
- pnpm (gestor de paquetes recomendado por velocidad y manejo de monorepo)

```
corepack enable
corepack prepare pnpm@latest --activate
```

### 1.2 Estructura del repositorio (monorepo simple)

```
boxadmin/
├── docker-compose.yml
├── .env.example
├── apps/
│   └── api/                                # NestJS backend
│       ├── src/
│       ├── prisma/
│       ├── test/
│       ├── package.json
│       └── tsconfig.json
├── packages/
│   └── shared/                             # DTOs y tipos compartidos (futuro frontend)
└── README.md
```

### 1.3 Bootstrap del backend NestJS

```
mkdir -p boxadmin/apps/api && cd boxadmin/apps/api
pnpm dlx @nestjs/cli new . --package-manager pnpm --skip-git
```

### 1.4 Dependencias a instalar (dentro de apps/api)

```
# Core de dominio
pnpm add @nestjs/config @nestjs/jwt @nestjs/passport passport passport-jwt
pnpm add @prisma/client argon2 class-validator class-transformer
pnpm add @nestjs/bullmq bullmq ioredis
pnpm add helmet compression

# Desarrollo
pnpm add -D prisma @types/passport-jwt
```

**Nota sobre hashing de contraseñas:** usar argon2 en vez de bcrypt — mejor resistencia a ataques por hardware y ya es el estándar recomendado en 2026. No usar bcryptjs.

### 1.5 docker-compose.yml (raíz del repo)

```
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: boxadmin
      POSTGRES_PASSWORD: boxadmin_dev
      POSTGRES_DB: boxadmin
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  api:
    build:
      context: ./apps/api
      env_file: .env
    ports:
      - "3000:3000"
    depends_on:
      - postgres
      - redis
    volumes:
      - ./apps/api:/app
      - /app/node_modules

volumes:
  pgdata:
```

## 1.6 Variables de entorno (.env.example)

```
DATABASE_URL=postgresql://boxadmin:boxadmin_dev@localhost:5432/boxadmin?schema=public
REDIS_URL=redis://localhost:6379
JWT_SECRET=changeme_dev_secret
JWT_EXPIRES_IN=1d
JWT_REFRESH_SECRET=changeme_dev_refresh_secret
JWT_REFRESH_EXPIRES_IN=30d
PORT=3000
```

---

## 2. Modelo de datos base (Prisma)

Inicializar Prisma:

```
pnpm dlx prisma init
```

prisma/schema.prisma — modelos de esta fase:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum RolUsuario {
  SUPERADMIN
  ADMIN_SALON
  ADMIN_OPERATIVO
  PROFESOR
  ALUMNO
  FANTASMA
}

model Tenant {
  id          String   @id @default(cuid())
  nombre      String
  slug        String   @unique
  activo      Boolean  @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  usuarios    Usuario[]

  @@map("tenants")
}

model Usuario {
  id          String   @id @default(cuid())
  tenantId    String
  tenant      Tenant   @relation(fields: [tenantId], references: [id])

  nombreCompleto String
  email         String
  passwordHash  String
  rol           RolUsuario
}
```

```

    activo          Boolean    @default(true)

    refreshTokens   RefreshToken[]

    createdAt DateTime @default(now())
    updatedAt DateTime @updatedAt

    // Un email es único DENTRO de un tenant, no globalmente
    @@unique([tenantId, email])
    @@index([tenantId])
    @@map("usuarios")
}

model RefreshToken {
  id          String    @id @default(cuid())
  usuarioId   String
  usuario     Usuario   @relation(fields: [usuarioId], references: [id], onDelete: Cascade)
  tokenHash   String
  expiresAt   DateTime
  revokedAt   DateTime?
  createdAt   DateTime @default(now())

  @@index([usuarioId])
  @@map("refresh_tokens")
}

model HistorialAccion {
  id          String    @id @default(cuid())
  tenantId    String
  usuarioId   String?
  entidad     String    // ej: "Turno", "Usuario", "Reserva"
  entidadId   String
  accion      String    // ej: "crear", "cancelar", "editar"
  detalle     Json?
  createdAt   DateTime @default(now())

  @@index([tenantId, entidad, entidadId])
  @@map("historial_acciones")
}

```

**Decisión de diseño clave:** tenant\_id como columna directa en cada tabla (shared schema / row-level multi-tenancy), no schema-per-tenant. Es la opción correcta para una v1: migraciones simples, un solo pool de conexiones, backups triviales. Reevaluar solo si en el futuro hay requisitos regulatorios de aislamiento físico de datos por cliente.

HistorialAccion se crea desde esta fase, aunque no se use todavía activamente: todas las fases siguientes deben escribir aquí en cada acción relevante (alta, baja, cancelación, reasignación). Diseñarlo como afterthought en la Fase 6 es mucho más caro.

Generar y aplicar la migración inicial:

```
pnpm dlx prisma migrate dev --name init
```

### 3. Aislamiento por tenant (row-level, aplicado en código)

No usar Row Level Security de Postgres en esta fase (agrega complejidad operativa que no se justifica todavía). En su lugar:

1. **Prisma Client Extension** (`src/prisma/tenant-scoped.extension.ts`) que intercepta cada query y, si el modelo tiene columna `tenantId`, inyecta automáticamente el filtro `where: { tenantId }` usando un `AsyncLocalStorage` que guarda el tenant de la request actual.
2. **TenantContextMiddleware**: lee el `tenantId` del JWT decodificado y lo setea en el `AsyncLocalStorage` al inicio de cada request, antes de que cualquier guard o controller corra.
3. **Regla dura de código**: ningún query directo a Prisma debe pasar `tenantId` a mano en el `where` — siempre debe venir de la extensión. Esto evita el error más común y peligroso de un sistema multi-tenant: una query que se olvida de filtrar y filtra datos entre gimnasios distintos.

Estructura de carpetas sugerida para esto:

```
src/
├── common/
│   ├── tenant/
│   │   ├── tenant-context.ts          # AsyncLocalStorage
│   │   ├── tenant.middleware.ts
│   │   └── tenant-scoped.extension.ts
│   └── guards/
│       ├── jwt-auth.guard.ts
│       └── roles.guard.ts
├── auth/
│   ├── auth.module.ts
│   ├── auth.controller.ts
│   ├── auth.service.ts
│   ├── strategies/jwt.strategy.ts
│   └── dto/
├── prisma/
│   └── prisma.service.ts
└── main.ts
```

## 4. Autenticación y autorización

### 4.1 Endpoints de esta fase

| Método | Ruta                        | Descripción                                                      |
|--------|-----------------------------|------------------------------------------------------------------|
| POST   | <code>/auth/tenants</code>  | Crear un tenant nuevo (solo uso interno/superadmin en esta fase) |
| POST   | <code>/auth/register</code> | Crear el primer usuario ADMIN_SALON de un tenant                 |
| POST   | <code>/auth/login</code>    | Login → devuelve access token (JWT corto) + refresh token        |
| POST   | <code>/auth/refresh</code>  | Renueva el access token usando el refresh token                  |
| POST   | <code>/auth/logout</code>   | Revoca el refresh token actual                                   |
| GET    | <code>/auth/me</code>       | Devuelve el usuario autenticado (para verificar el guard)        |

### 4.2 Payload del JWT

```
{
  "sub": "usuario_id",
  "tenantId": "tenant_id",
}
```

```
"rol": "ADMIN_SALON",
"iat": 0,
"exp": 0
}
```

### 4.3 Guards y decorators a implementar

- `@Roles(...roles: RolUsuario[])` decorator + `RolesGuard`: valida el rol del JWT contra los roles permitidos en cada endpoint.
- `@CurrentUser()` decorator: extrae el usuario decodificado del request.
- `JwtAuthGuard` global (aplicado en `main.ts` vía `APP_GUARD`), con `@Public()` decorator para excluir rutas como `/auth/login`.

### 4.4 Jerarquía de roles (de esta especificación)

SUPERADMIN > ADMIN\_SALON > ADMIN\_OPERATIVO > PROFESOR > ALUMNO, más FANTASMA como rol técnico sin login real (usuario placeholder, no se loguea nunca — se crea directamente vía servicio interno en fases posteriores).

---

## 5. Infraestructura de jobs (para fases futuras)

Configurar BullMQ desde esta fase, aunque no haya jobs reales todavía, para no tener que introducir la dependencia a mitad de proyecto:

```
// app.module.ts
BullModule.forRoot({
  connection: {
    url: process.env.REDIS_URL,
  },
}),
```

Crear un módulo vacío `JobsModule` con una cola de ejemplo (`health-check-queue`) solo para validar que Redis y BullMQ están correctamente conectados.

---

## 6. Checklist de aceptación de la Fase 0

- ☐ `docker compose up` levanta Postgres, Redis y la API sin errores.
  - ☐ `prisma migrate dev` corre limpio y genera las tablas `tenants`, `usuarios`, `refresh_tokens`, `historial_acciones`.
  - ☐ Se puede crear un tenant vía `POST /auth/tenants`.
  - ☐ Se puede registrar un ADMIN\_SALON para ese tenant vía `POST /auth/register`.
  - ☐ `POST /auth/login` devuelve `access + refresh token` válidos.
  - ☐ `GET /auth/me` con el `access token` devuelve los datos correctos del usuario y su tenant.
  - ☐ Crear un segundo tenant y un segundo usuario, y verificar manualmente (con una query cruzada) que el `TenantContextMiddleware` + la extensión de Prisma impiden ver datos del otro tenant.
  - ☐ `RolesGuard` bloquea correctamente un endpoint de prueba marcado con `@Roles('ADMIN_SALON')` cuando lo llama un usuario con rol ALUMNO.
  - ☐ Existe al menos un test e2e de `auth` (registro + login + acceso a ruta protegida).
-

## 7. Qué NO entra en esta fase

Para mantener el scope acotado: no se implementan todavía Salas, Turnos, Reservas, Packs ni ninguna entidad de dominio del gimnasio — eso es el contenido de la **Fase 1**. Esta fase es puramente infraestructura y seguridad.